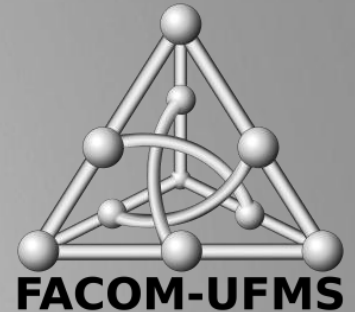




FUNDAÇÃO
UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



GERÊNCIA DE CONFIGURAÇÃO DE SOFTWARE

MÓDULO 4 - SISTEMAS DE GERENCIAMENTO DE CONFIGURAÇÃO DE SOFTWARE

Profa. Dra. Jane Sandim Eleutério

jane.eleuterio@ufms.br

O QUE VAMOS APRENDER HOJE?



- Ferramentas de GCS: Do Centralizado ao Distribuído
- Objetivos do Módulo
 - Diferenciar as arquiteturas de controle de versão (Centralizada vs. Distribuída)
 - Compreender a evolução histórica das ferramentas de GCS
 - Conhecer o ecossistema de ferramentas CASE de apoio ao ciclo de vida
 - Estabelecer critérios para a escolha da ferramenta ideal conforme o cenário do projeto

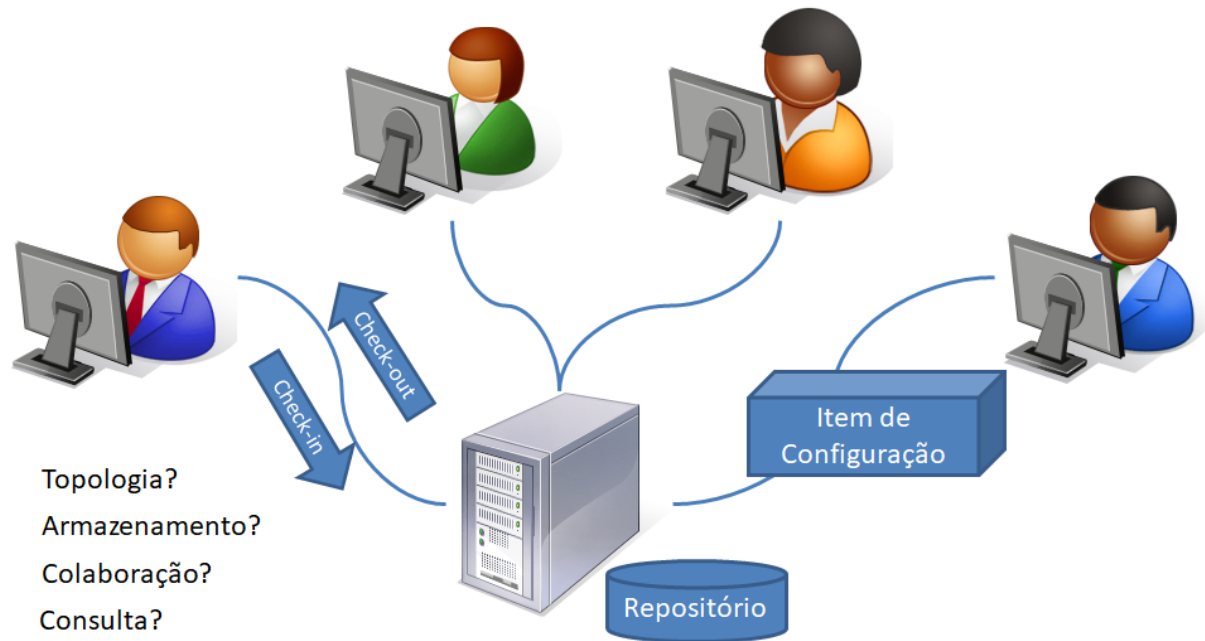
RECAPITULAÇÃO: A PONTE ENTRE A NORMA E A PRÁTICA

- **Módulo 3:** Vimos o PGCS (Plano de Gerência de Configuração) como a bússola do projeto, baseada na norma IEEE 828
- **Módulo 4:** Agora, saímos da teoria normativa para a realidade tecnológica
- **Reflexão:** "A ferramenta escolhida molda a cultura da equipe". Ela determina como os desenvolvedores colaboram e como a integridade é mantida



CONTROLE DE VERSÃO

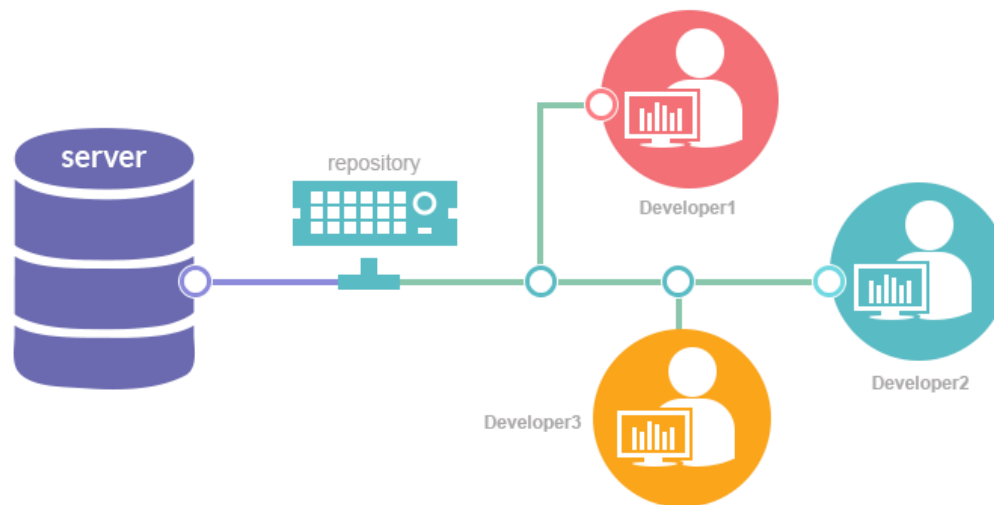
CONTROLE DE VERSÃO



COMO FUNCIONA O CONTROLE DE VERSÃO?

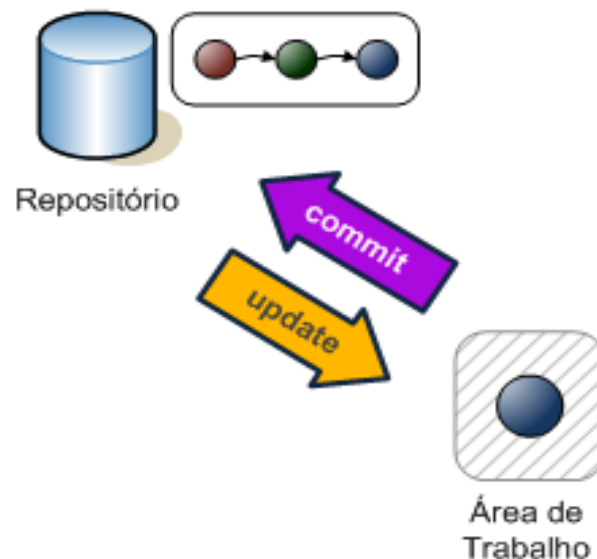
➤ O controle de versão é composto de duas partes:

- Repositório
- Área de Trabalho



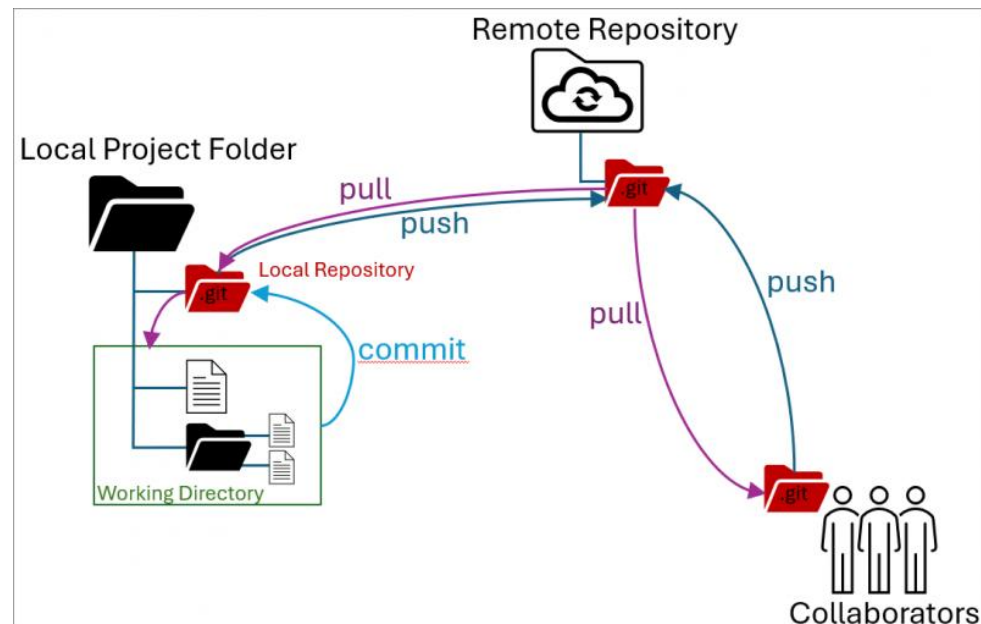
CONTROLE DE VERSÃO: REPOSITÓRIO

- Armazena todo o histórico de evolução do projeto
- Registra toda e qualquer alteração feita em cada item versionado
- Contém os arquivos versionados e os *commits*, ou seja, os registros de mudanças feitas ao longo do tempo.
- O desenvolvedor não trabalha diretamente nos arquivos do repositório



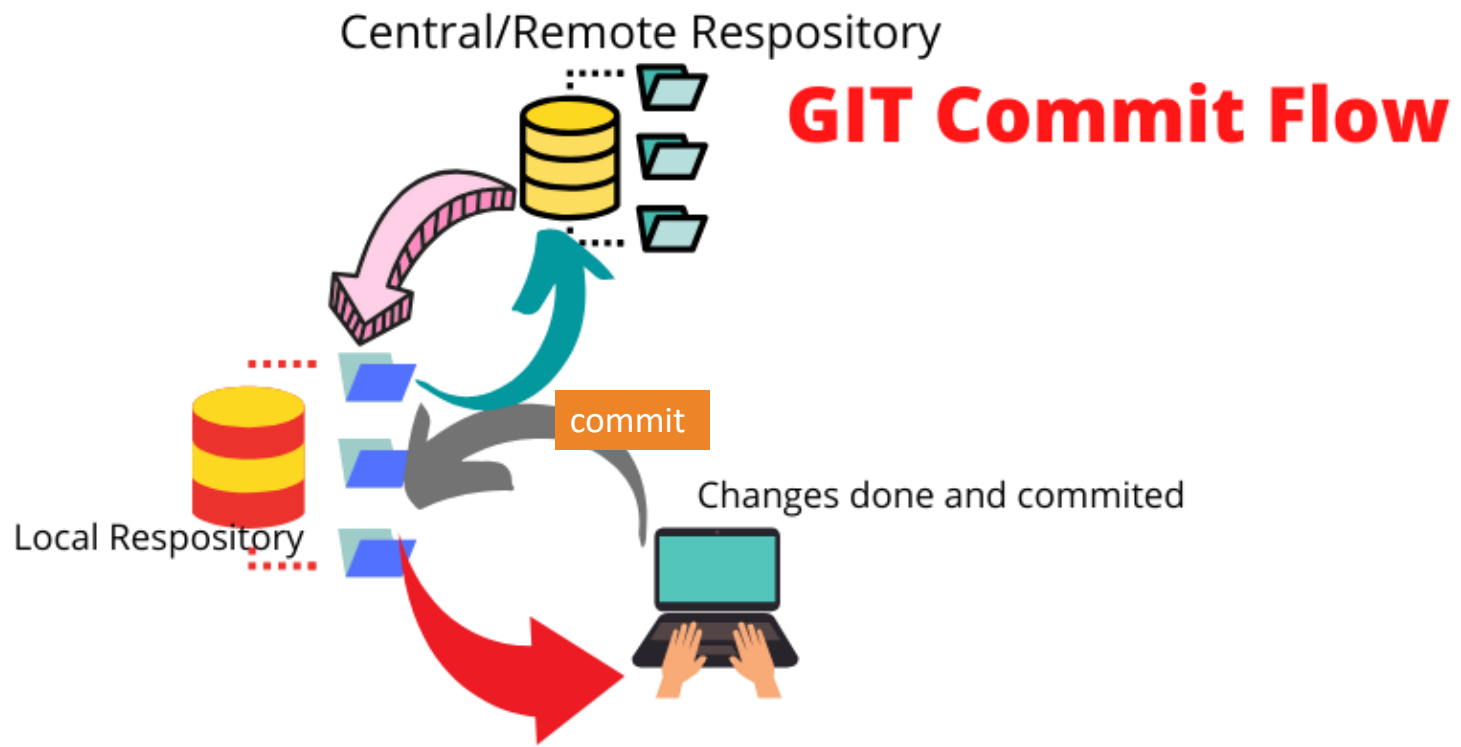
CONTROLE DE VERSÃO: ÁREA DE TRABALHO

- Contém a cópia dos arquivos do projeto e que é monitorada para identificar as mudanças realizadas
- É individual e isolada das demais áreas de trabalho
- É o **espaço local** no computador do desenvolvedor onde os arquivos são editados



COMMIT

- Commit é o registro permanente de um conjunto de alterações feitas em arquivos, criando uma nova revisão no repositório
- Um commit é o ato de salvar, de forma permanente, um conjunto de alterações em um arquivo ou conjunto de arquivos no seu sistema de controle de versão



COMMIT

- Representa o encerramento de uma unidade lógica de trabalho e permite rastreabilidade, reversão e colaboração no desenvolvimento de software

```
git commit -m "refactor: changed the markup"
git commit -m "refactor: change the markup"
```

The screenshot displays a GitHub pull request interface. At the top, there are tabs for 'Conversation' (4), 'Commits' (1), 'Checks' (52), and 'Files changed' (34). Below these, a summary bar shows 'Changes from all commits', a 'File filter', 'Conversations', and a 'Review changes' button. The left sidebar shows a file tree with folders 'content', 'actions', 'automating-builds-and-tests', and 'creating-actions'. The 'automating-builds-and-tests' folder is expanded, showing files like 'building-and-testing-jav...', 'building-and-testing-no...', 'building-and-testing-ru...', and 'building-and-testing-sw...'. The main area shows a diff view for the file '...nt/actions/automating-builds-and-tests/building-a...'. The diff highlights changes in the 'jobs' section, specifically the 'Validate Gradle wrapper' job, which has been updated with a new action ID.

Conversation 4 Commits 1 Checks 52 Files changed 34 +80 -80

Changes from all commits File filter Conversations 0 / 34 files viewed Review changes

Filter changed files

- content
 - actions
 - automating-builds-and-tests
 - building-and-testing-jav...
 - building-and-testing-no...
 - building-and-testing-ru...
 - building-and-testing-sw...
 - creating-actions

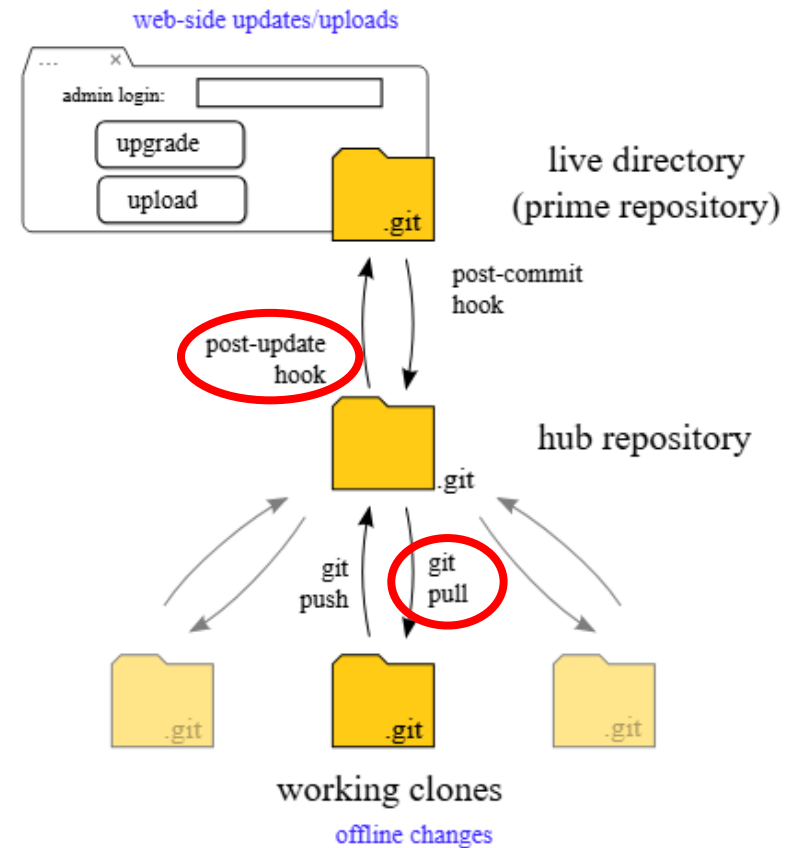
...nt/actions/automating-builds-and-tests/building-a...

@@ -69,7 +69,7 @@ jobs:

69	69	java-version: '17'
70	70	distribution: 'temurin'
71	71	- name: Validate Gradle wrapper
72	-	uses: gradle/wrapper-validation-
		action@e6e38bacfd1a337459f332974bb2327a31aaf4b
72	+	uses: gradle/wrapper-validation-
		action@094d8354f0cc239ee6a315e3331528fbc2008041
73	73	- name: Build with Gradle
74	74	uses: gradle/gradle-build-
		action@67421db6bd0bf253fb4bd25b31ebb98943c375e1

UPDATE

- É o processo de **atualizar a cópia de trabalho local** (Área de Trabalho) com as alterações mais recentes que foram ***committadas*** no **repositório central** por outros desenvolvedores
- No Git:
 - Push: Envia as alterações locais para o repositório remoto (git push)
 - Pull: Traz as alterações do repositório remoto e mescla com o local (git pull)





A EVOLUÇÃO DAS FERRAMENTAS

A EVOLUÇÃO DAS FERRAMENTAS

➤ **Anos 80 e 90:** O controle era como uma biblioteca física (modelo de reserva de arquivos)

- Controle de arquivos individuais em uma única máquina
- Apenas um usuário por vez no arquivo (bloqueio local)

- Source Code Control System (SCCS)
- Revision Control System (RCS)

É um dos sistemas mais antigos e fundamentais de GCS, focado especificamente no controle de versões de arquivos individuais. Criado nos primórdios do Unix, ele estabeleceu as bases para ferramentas mais modernas, permitindo rastrear mudanças, gerenciar versões e controlar quem pode alterar o código.

A EVOLUÇÃO DAS FERRAMENTAS

➤ **Anos 2000:** O auge do Centralizado

- Surge a necessidade de servidores para colaboração em equipe
- Servidor central, permitindo múltiplos usuários, mas dependente de conexão
- Concurrent Versions System (CVS)
- Apache Subversion (SVN)



A EVOLUÇÃO DAS FERRAMENTAS

- **Era Moderna:** O domínio do Distribuído
 - Foco em paralelismo, velocidade e trabalho offline
 - Repositórios locais completos e alta velocidade de ramificação (*branching*)
 - Git, Mercurial, Bitkeeper



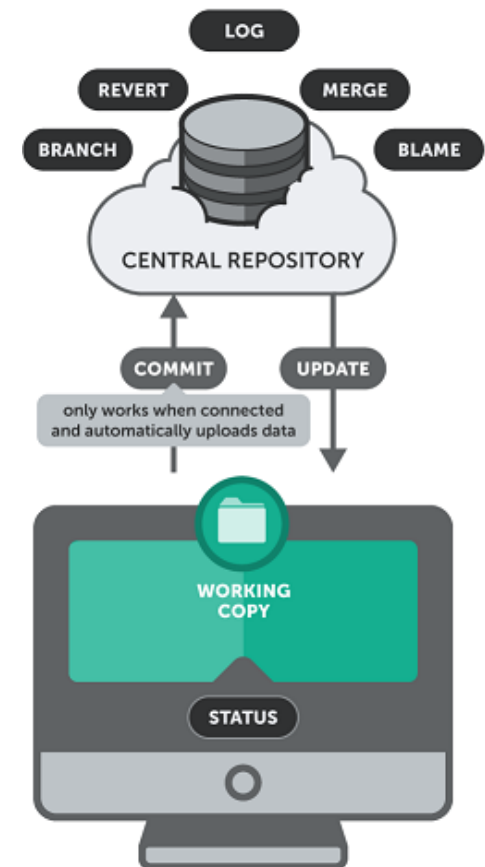
SISTEMAS CENTRALIZADOS

SISTEMAS CENTRALIZADOS (CVCS)

- **O Modelo:** Existe um único servidor central que guarda todo o histórico das versões
- **Funcionamento:**
 - O desenvolvedor faz o checkout de uma "*Working Copy*" (cópia de trabalho)
 - Cada alteração (*commit*) é enviada diretamente ao servidor
 - Para ver o histórico ou comparar versões, é necessária conexão com o servidor

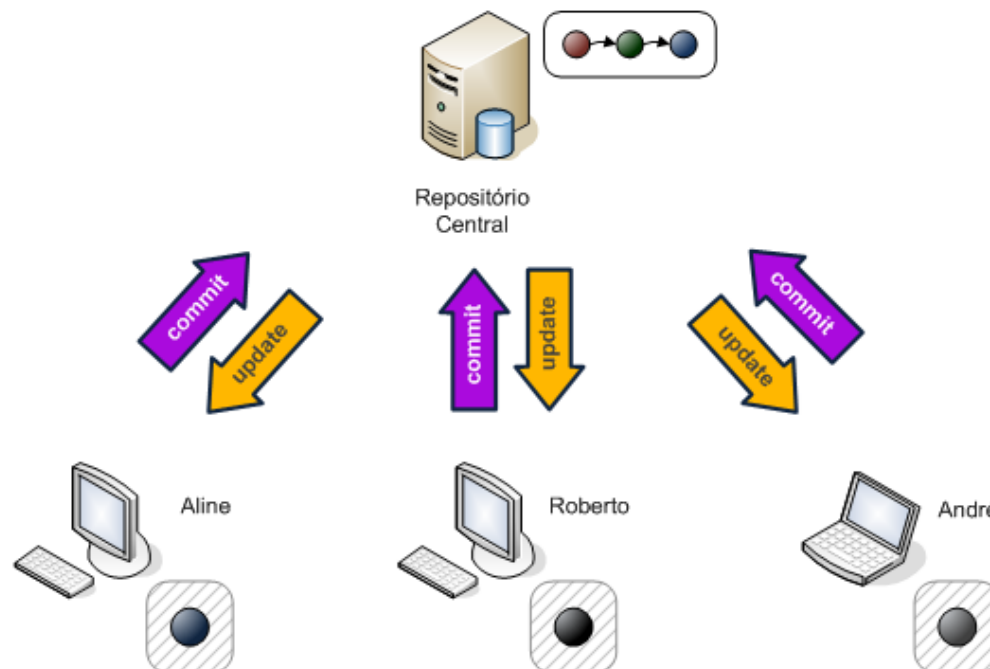
SISTEMAS CENTRALIZADOS (CVCS)

- Nessa configuração, existe um único repositório central armazenado em um servidor, e os desenvolvedores acessam esse servidor para obter a versão mais recente dos arquivos.
- Após modificarem os arquivos localmente, eles realizam um commit para atualizar o repositório central, tornando as alterações visíveis para os demais desenvolvedores



SISTEMAS CENTRALIZADOS (CVCS)

- No controle de versão centralizado há um único repositório e várias cópias de trabalho que se comunicam apenas através do repositório central
- É considerado como uma fonte autoritária: todas as operações de versionamento são realizadas nele



SISTEMAS CENTRALIZADOS (CVCS)

- Mecanismo CVCS: **Lock-Modify-Unlock**
- Alguns sistemas centralizados (como o Perforce) utilizam o bloqueio de arquivos
- **O fluxo:**
 - O usuário A "bloqueia" o arquivo (ninguém mais edita)
 - O usuário A altera e envia
 - O usuário A desbloqueia
- **Vantagem:** Evita conflitos de mesclagem (*merges*) complexos em arquivos binários (ex: modelos 3D, imagens)

CVCS: VANTAGENS E DESAFIOS

➤ Vantagens:

- Controle Administrativo: Facilidade em definir permissões rígidas (ex: quem pode ler/escrever em pastas específicas)
- Arquivos Grandes: Melhor desempenho com arquivos binários pesados

➤ Desafios:

- Ponto Único de Falha: Se o servidor cair, ninguém faz *commit* ou acessa o histórico
- Dependência de Rede: Lentidão em operações frequentes por depender de conexão constante

EXEMPLO CLÁSSICO: APACHE SUBVERSION (SVN)

- **Centralização:** O SVN utiliza um servidor central. Os usuários possuem réplicas locais apenas das versões atuais, enviando alterações diretamente para o servidor
- **Controle de Versão** (Revisões): A cada commit (envio de alterações), o SVN cria um número de revisão sequencial, armazenando todo o histórico
- **Fluxo de Trabalho:** Baseia-se em *checkout* (baixar o projeto), modificação local, resolução de conflitos e *commit* (enviar as alterações)
- **Estrutura:** Utiliza *trunk* (desenvolvimento principal), *branches* (ramos) e *tags* (etiquetas de versão) para organizar o código

O trunk é o ramo de desenvolvimento principal.
Branches são ramos de desenvolvimento paralelos.

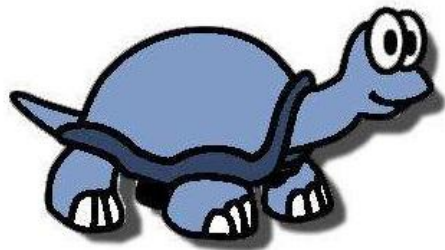


EXEMPLO CLÁSSICO: APACHE SUBVERSION (SVN)

➤ Vantagens do SVN:

- Facilidade para gerenciar grandes volumes de dados ou arquivos binários
- Mais simples para usuários que preferem um modelo de controle centralizado
- Ferramentas visuais como o TortoiseSVN facilitam a visualização do histórico e conflitos

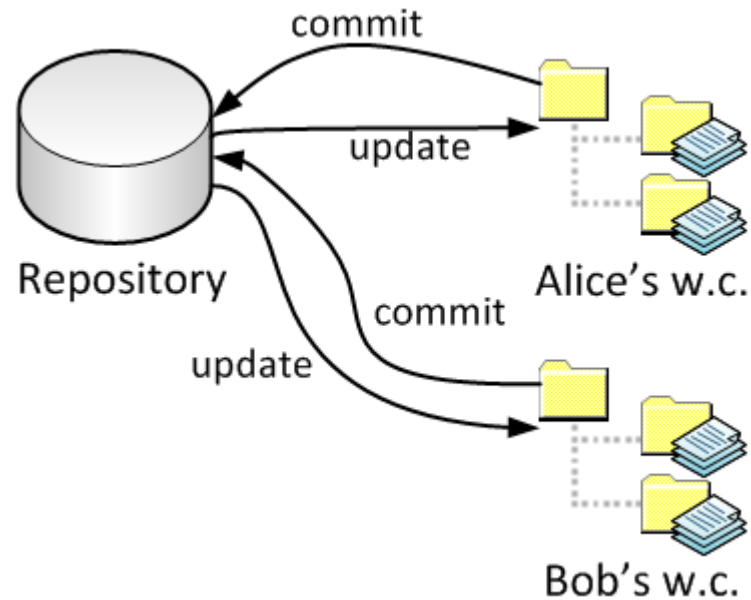
➤ O Subversion foi projetado para ser um substituto moderno e mais robusto do antigo CVS (Concurrent Versions System)



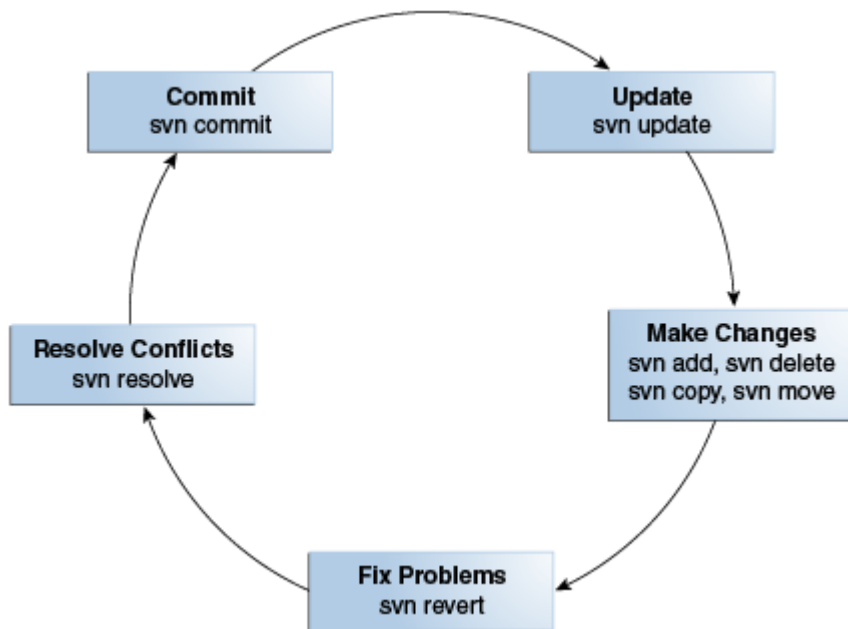
TortoiseSVN



EXEMPLO CLÁSSICO: APACHE SUBVERSION (SVN)



EXEMPLO CLÁSSICO: APACHE SUBVERSION (SVN)



- Criar working copy (cópia de trabalho):
 - svn checkout
- Analisar suas mudanças:
 - svn status
 - svn diff
- Atualizar sua working copy:
 - svn update
- Resolver conflitos (merge com mudanças de outros):
 - svn update
 - svn resolve
- Fazer mudanças:
 - svn add
 - svn delete
 - svn copy
 - svn move
- Enviar / efetivar suas mudanças:
 - svn commit
- Desfazer algumas mudanças (undo):
 - svn revert



SISTEMAS DISTRIBUÍDOS

SISTEMAS DISTRIBUÍDOS (DVCS)

- Exemplo: Git
- **O Modelo:** Cada máquina de desenvolvedor é um repositório completo
- **Funcionamento:**
 - O Clone copia todo o histórico do projeto, não apenas a última versão
 - Os *Commits* são locais e instantâneos
 - A sincronização com o time ocorre via *Push* e *Pull*

SISTEMAS DISTRIBUÍDOS (DVCS)

➤ **Peer-to-Peer:**

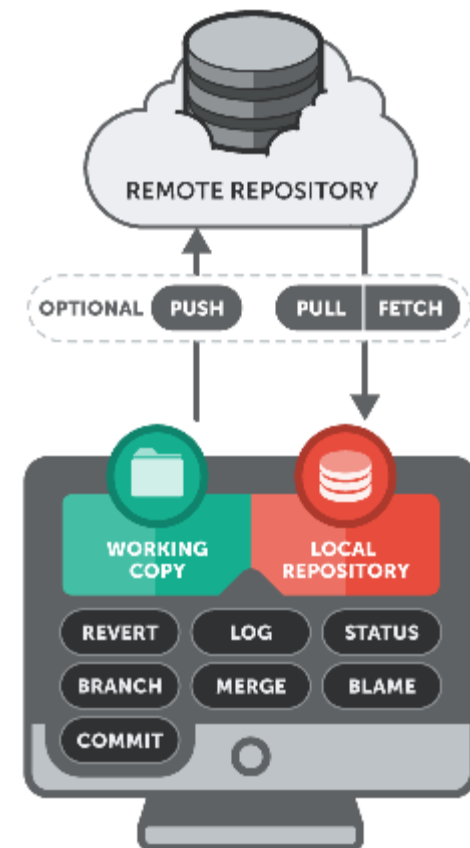
- Não existe apenas um repositório "mestre" na arquitetura técnica; cada cópia de trabalho é um repositório completo
- Cada desenvolvedor possui em sua máquina uma cópia completa do repositório, incluindo todo o histórico de versões

➤ **Snapshots vs. Diff:**

- Diferente dos antigos que guardavam "mudanças de texto", sistemas modernos como o Git guardam "fotos" (instantâneos) do estado do projeto

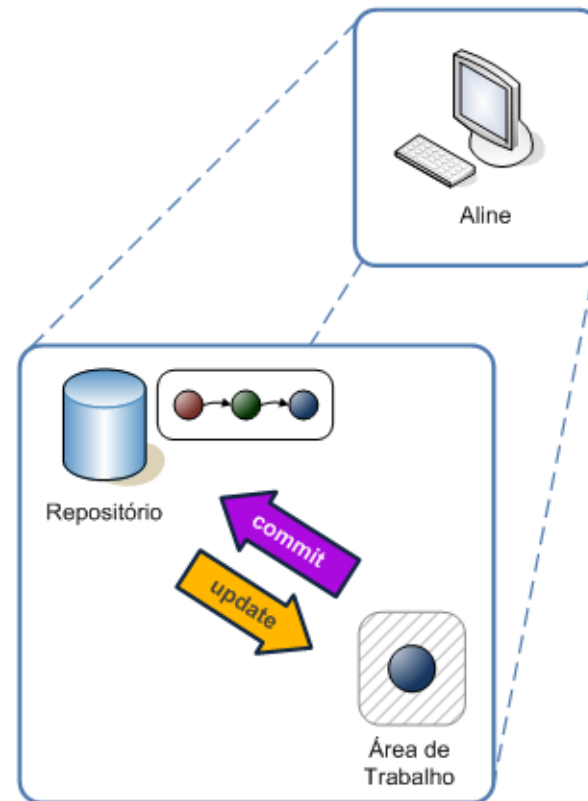
SISTEMAS DISTRIBUÍDOS (DVCS)

- Permite que os desenvolvedores trabalhem de forma independente e até mesmo offline, realizando *commits* localmente e sincronizando com os repositórios de outros desenvolvedores quando necessário

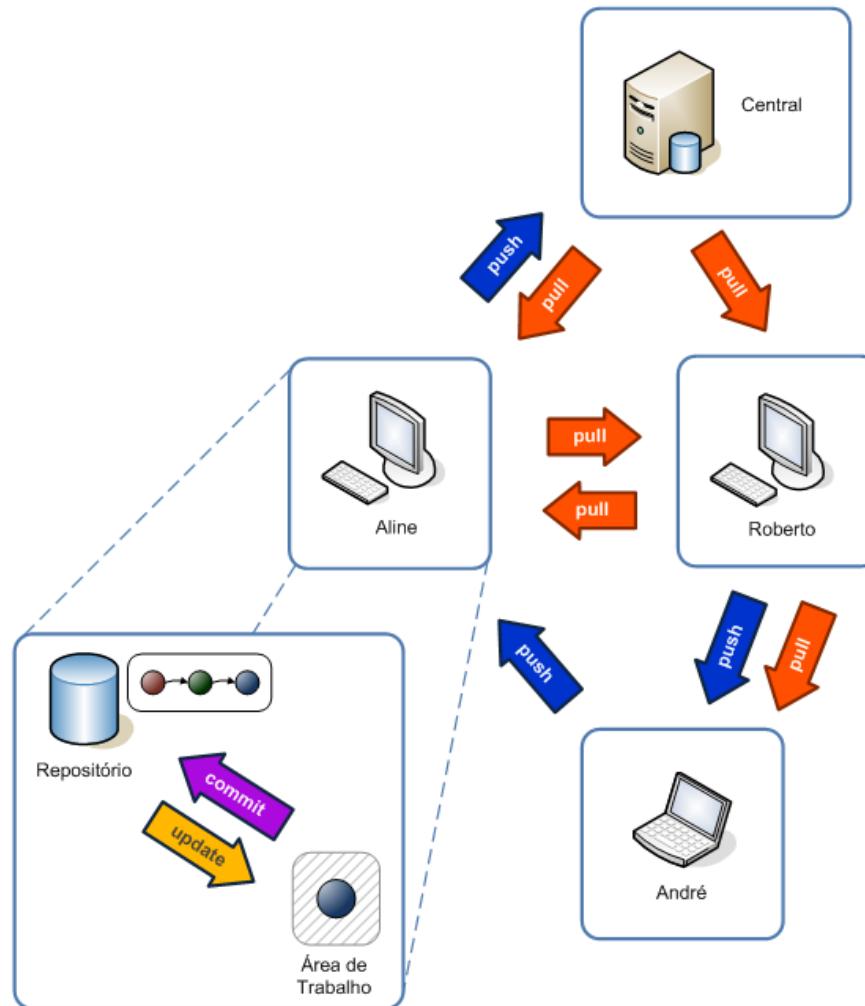


SISTEMAS DISTRIBUÍDOS (DVCS)

- No controle de versão distribuído cada desenvolvedor possui um repositório próprio acoplado a uma área de trabalho.



SISTEMAS DISTRIBUÍDOS (DVCS)



DVCS: VANTAGENS

➤ Vantagens do Ponto de Vista do Desenvolvedor:

- **Rapidez:** Quase todas as operações são locais e extremamente rápidas
 - Operações Locais: log, diff, commit e branch ocorrem em milissegundos, sem internet
- **Autonomia:** Possibilidade de trabalhar e manter o histórico completo sem internet

DVCS: VANTAGENS

➤ Vantagens do Ponto de Vista da Gerência/Coordenação:

▪ Confiabilidade:

- No sistema centralizado, uma pane no servidor interrompe todo o desenvolvimento. Já no sistema distribuído, além de a equipe poder continuar seu trabalho, os repositórios dos desenvolvedores funcionam como cópias de backup de todo o projeto
- Cada cópia de desenvolvedor funciona como um backup completo do servidor

▪ Redução de custos com servidor:

- A carga de processamento fica distribuída nas próprias máquinas dos desenvolvedores. O repositório "central", quando existe, tem o papel do repositório "oficial" e não como processador central das requisições.

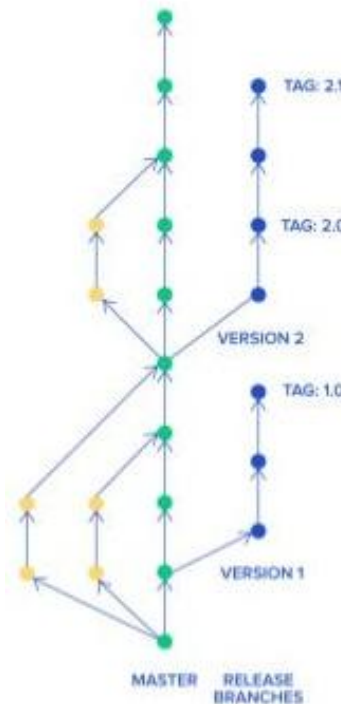
▪ Flexibilidade de Fluxo:

- Permite fluxos de trabalho variados (GitFlow, Trunk-based), essenciais para DevOps

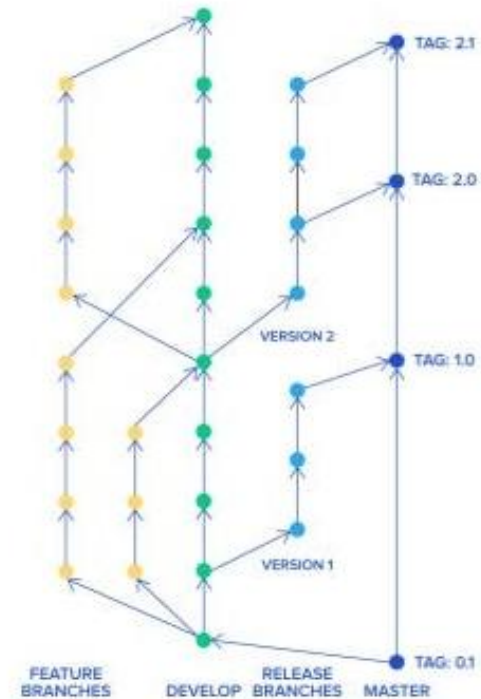
FLUXO DE TRABALHO E BRANCHING MODEL

- Sistemas distribuídos, como o Git, permitem que todos tenham o histórico completo. Mas se todo mundo começar a criar ramos sem critério, o projeto vira um labirinto. No Módulo 6, vamos apresentar o GitFlow, que é a “etiqueta social” e o processo que as grandes empresas usam para organizar essa bagunça
- Veremos sobre fluxo de trabalho e branching model no módulo 6

Trunk-Based



GitFlow



DVCS: VANTAGENS E DESAFIOS

➤ Desafios:

- **Curva de Aprendizado:** Alguns conceitos podem ser complexos inicialmente
- **Espaço em Disco:** Repositórios com histórico muito longo e arquivos binários podem se tornar pesados



CONCLUSÕES E FERRAMENTAS

COMPARATIVO: CENTRALIZADO VS. DISTRIBUÍDO

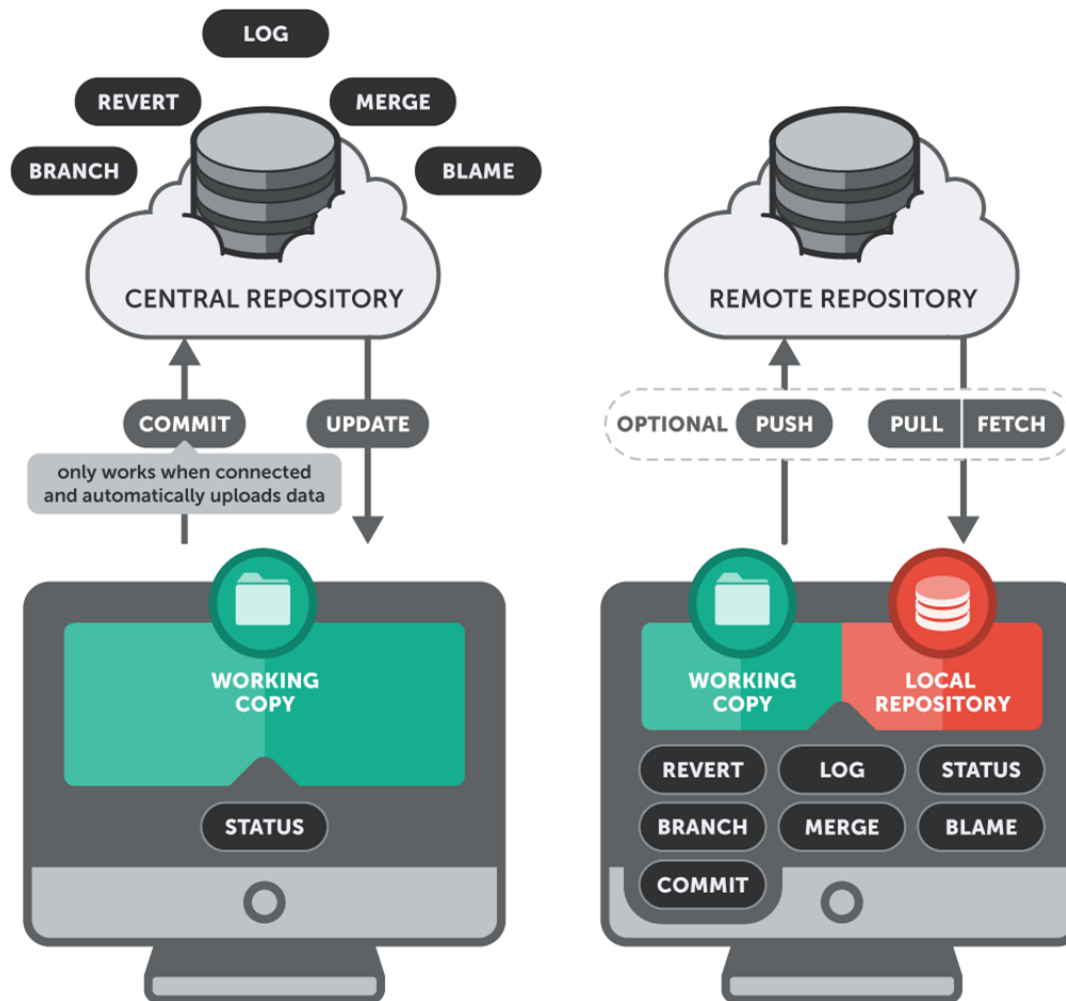
Característica	Centralizado	Distribuído
Principal exemplo	SVN	Git
Local do histórico	Servidor remoto	Local em cada máquina de desenvolvedor
Conexão	Exige internet para quase tudo	Internet apenas para sincronizar
Risco	Servidor fora = Equipe parada	Se o servidor cair, o backup está com os devs
Ramificação (Branching)	Pesada / Cara	Leve / Barata
Curva de aprendizado	Baixa / Média	Média / Alta

COMPARATIVO: CENTRALIZADO VS. DISTRIBUÍDO

➤ Operações Básicas dos Controles de Versão Centralizado e Distribuídos

Centralizado	Distribuído	Descrição
checkout	clone	Criação da cópia de trabalho/repositório
commit	commit	Envia alterações para o repositório, criando uma revisão
update	update	Atualiza a cópia/área de trabalho em uma revisão
	pull	Importa revisões feita em outro repositório
	push	Envia revisões locais para outro repositório

COMPARATIVO: CENTRALIZADO VS. DISTRIBUÍDO



QUANDO USAR CADA UM?

➤ Use Centralizado se:

- O projeto lida com arquivos binários gigantescos (Gigas de texturas de jogos)
- O controle de acesso precisa ser extremamente granular (ex: o Dev X só pode ver a pasta /A)

➤ Use Distribuído se:

- A equipe é grande e precisa de agilidade
- O foco é código-fonte (texto)
- O projeto exige integração contínua (CI/CD) veloz

FERRAMENTAS DE CONTROLE DE VERSAO

Software Livre	Software Comercial
→ Git → Mercurial → Subversion	→ BitKeeper (BitMover) → ClearCase (IBM Rational) → Perforce → PVCS (Serena) → StarTeam (Borland) → Synergy/CM (Telelogic) → Team Foundation Server (Microsoft)

FERRAMENTAS CASE DE APOIO

- A GCS não vive só de controle de versão. Ela se integra ao ecossistema CASE
- A GCS moderna utiliza um conjunto de ferramentas integradas para garantir a qualidade
 - **Gestão de Mudanças:**
 - Jira, Redmine, Bugzilla, Trello
 - Rastreamento de bugs e tarefas
 - **Repositórios na Nuvem (SaaS):**
 - GitHub, GitLab, Bitbucket, Azure DevOps
 - **Code Review:**
 - Ferramentas integradas para revisão de código antes da integração na base principal

INTEGRAÇÃO: A "COLA" DO PROJETO

➤ Rastreabilidade Vertical:

- É possível configurar a ferramenta para que um *commit* só seja aceito se mencionar o número de um "Ticket" de mudança (Módulo 5)
- Exemplo: "Commit 452a - Correção do bug #102"
- Isso cria uma trilha de auditoria (Módulo 7)

FERRAMENTAS DE BUILD E QUALIDADE

➤ Para garantir que os Itens de Configuração (ICs) integrados são válidos:

- **Automação de Build:**

- Maven (Java), Gradle (Android), NPM (JS)
- Transformar código em executável automaticamente

- **Análise Estática:**

- SonarQube
- Auditoria automática de código

➤ Onde entram na GCS?

- Elas verificam se a nova configuração do software não "quebrou" o que já estava funcionando

ESCOLHENDO A FERRAMENTA CERTA

➤ O que considerar ao decidir?

1. Perfil da Equipe: Experiência técnica e distribuição geográfica
2. Tipo de Projeto: Código puro vs. Muitos ativos binários (jogos, design)
3. Necessidade de Controle: Níveis de permissão e governança exigidos
4. Integração: Compatibilidade com ferramentas de CI/CD (Módulo 6)

CONCLUSÃO

➤ Resumo:

- Sistemas distribuídos ganharam o mercado pela flexibilidade, mas o modelo centralizado ainda tem nichos importantes (indústrias específicas)
- **Próxima Aula:** Módulo 5 – O Processo de Controle de Mudanças (Como gerenciar o caos das solicitações de alteração)

REFERÊNCIAS BIBLIOGRÁFICAS

- HASS, Anne Mette Jonassen. **Configuration management principles and practice**. Boston, MA: Addison-Wesley, c2003. xlv, 370 p. (The Agile software development series). ISBN 0321117662.
- LEON, Alexis. **Software configuration management handbook**. 2. ed. Boston, MA: Artech House, c2005. xxiii, 383 p. ISBN 9781580538824.
- PRESSMAN, Roger S. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre, RS: AMGH Ed., 2016. xxviii, 940 p. ISBN 9788580555332.
- VALENTE, Marco Tulio. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. 1ª. ed.: Independente, 2022. Disponível online em <https://engsoftmoderna.info/>
- SOMMERVILLE, Ian. *Engenharia de Software*, 2018.